

01_MASTER_PROJECT_SPECIFICATION

Engineering source of truth for the BorderPulse / AI multi-sensor restricted-area surveillance prototype.

1. Objective

Build a working, modular, low-latency restricted-area security prototype. Current target: Windows laptop + built-in camera + YOLO + polygon restricted zone + ESP32-S development board + buzzer + Supabase. Future target: Raspberry Pi 5 + camera + real radar/ground sensors.

Primary goal: detect people entering a user-defined restricted area while reducing false alarms using object classification, tracking, temporal validation and optional sensor evidence.

2. Engineering corrections that must not be violated

Do not require camera + radar + ground as a strict AND rule for every alarm. Use graded evidence and temporal confirmation.

RCWL-0516 is motion/presence evidence; it does not reliably provide target coordinates, angle or identity. Do not claim it can point a camera to an exact object.

YOLO object detection alone does not prove fighting or abnormal behaviour. Use measurable secondary analytics such as people count, proximity, speed, direction, dwell time and repeated boundary crossing. A dedicated behaviour model can be added later.

Do not store large video binaries in PostgreSQL. Store media in Supabase Storage and metadata in Postgres.

Never block camera/UI processing on YOLO, database operations, video encoding or cloud uploads.

3. Current and future hardware

Current: Windows laptop, laptop built-in camera, ESP32-S development board, buzzer.

Future: Raspberry Pi 5, USB/CSI camera, RCWL-0516 or improved radar, ground/vibration sensor, optional environmental sensor, optional PTZ/thermal camera.

Current runtime: `RUNTIME_MODE=laptop` and `SENSOR_SIMULATION=true`. Future runtime: `RUNTIME_MODE=raspberry_pi`.

4. System architecture

Camera → frame capture → preprocessing → YOLO detection → object tracking → restricted-zone engine → behaviour/activity analytics → camera-health engine → ESP32 sensor state → temporal sensor fusion → decision engine → UI alert + ESP32 buzzer command + snapshot + evidence video + database/storage.

Use provider interfaces so CameraProvider, InferenceProvider, Esp32Provider, StorageProvider and EventRecorder can be replaced without rewriting the decision engine.

5. YOLO and tracking

Use a current Ultralytics YOLO model, initially YOLO11n or compatible. Support pretrained COCO classes including person, bird, common animals and vehicles. Allow a custom model later.

Use ByteTrack or the tracking mechanism supported by the installed Ultralytics version. Every detection should include class, confidence, bounding box, `track_id` and timestamp.

Do not claim pretrained COCO recognizes arbitrary fighting, military activity or suspicious behaviour.

6. Restricted zone

Use a polygon, not only a rectangle. Operator clicks three or more vertices on the live camera, then finishes the polygon. Vertices must be editable, draggable and deletable.

Store normalized coordinates from 0 to 1 so the zone survives camera resolution changes. For intrusion, use the tracked person's bottom-center/foot point for the point-in-polygon test.

7. Alert logic

High-priority human intrusion: person confidence $\geq$ configurable high threshold (default 0.85) AND person is inside the restricted polygon → immediate critical alert, ESP32 buzzer, dashboard alert, snapshot and evidence video.

Lower-confidence detections use temporal confirmation plus optional radar/ground evidence. Initial configurable evidence weights may be vision 0.55, radar 0.20, ground 0.15 and temporal 0.10; label these as initial engineering values, not scientifically validated constants.

Animal/bird/vehicle detections are displayed/logged and do not activate the intrusion buzzer by default.

Implement event states: NO_DETECTION → POSSIBLE_DETECTION → TEMPORAL_CONFIRMATION → CONFIRMED → ALARM_ACTIVE → EVIDENCE_CAPTURE → EVENT_ACTIVE → EVENT_RESOLVED, with FALSE_POSITIVE support.

Use event cooldown and duplicate suppression so one continuous intrusion does not create hundreds of events.

8. Temporal validation and evidence

Default temporal validation: minimum 3 frames within a configurable 1-second window for normal intrusion confirmation. A very high-confidence person clearly inside the zone may escalate immediately.

Use a rolling 3–5 second pre-event buffer. On trigger, preserve the prebuffer, capture a full-frame snapshot and cropped evidence image, then record approximately 5–10 seconds after trigger. Workflow: PREBUFFER → TRIGGER → SNAPSHOT → CONTINUE RECORDING → FINALIZE CLIP → UPLOAD → DB METADATA.

Current webcam supports digital crop/zoom only. Do not claim optical zoom. Create PTZ adapter methods for a future PTZ camera.

9. Radar and sensor mapping

RCWL-0516 should be mapped manually to a named camera zone. If radar triggers, investigate that zone. With a fixed webcam, use digital crop/zoom in 0.5–1.0 second steps if desired; physical pan/tilt is future hardware.

Ground sensor evidence is treated as GROUND_TRIGGER, not HUMAN_DETECTED. Real sensor electrical specifications must be verified before wiring.

10. ESP32

ESP32 responsibilities: read future sensors, control buzzer, send sensor state, receive alarm commands and report health. YOLO runs on the laptop/Raspberry Pi, not on the ESP32.

Primary prototype communication: Wi-Fi + HTTP REST; USB serial can be a bench-testing fallback. Example endpoints: GET /api/status, GET /api/sensors, POST /api/buzzer, POST /api/config. Implement heartbeat and an explicit ESP32 OFFLINE state.

IMPORTANT: the exact board must be verified before physical wiring. The user-provided board is labeled ESP32S, but do not assume its chip/pinout without board verification.

11. Provisional pin interface

Use a configurable pin map rather than hard-coding pins throughout the application. Earlier project documents proposed GPIO25 buzzer, GPIO26 ground, GPIO27 radar and GPIO34 optional analog/environment input for a classic ESP32-style board. A separate ESP32-S3 mapping was also proposed in earlier material. Therefore Antigravity must verify the actual board/chip and freeze the correct pin map before wiring.

Never apply a 5 V signal directly to an ESP32 GPIO. All sensor output voltage levels must be checked. Common ground is required where electrically appropriate.

12. Supabase

Use the already-connected Supabase project. Inspect its existing schema before modifying anything. Expected tables from the project specification: devices, cameras, zones, sensor_readings, detections, events, event_media, camera_health and system_settings.

Use private Storage buckets such as event-images, event-videos and optionally event-clips. Store large media in Storage and metadata in event_media/Postgres.

Never expose a service-role key in the frontend. Keep secrets in environment variables. Keep RLS enabled.

13. Frontend

Use React + Vite + TailwindCSS and use the connected Stitch/Figma design as the visual source of truth. Pages: Overview, Live Monitor, Events, Event Details, Zones, Sensors, Devices, Camera Health, Analytics and Settings.

Live Monitor must show low-latency video, bounding boxes, labels, confidence, track IDs, restricted polygon, sensor states and current threat state. Event investigation must show snapshot, video, metadata and sensor evidence.

Do not add unnecessary animation or effects that reduce FPS.

14. Performance

Separate camera capture, inference, UI rendering, recording and uploads. Use bounded queues and drop stale frames when overloaded. Always display the latest available frame. Run uploads and encoding in background workers. Show FPS, inference latency, queue depth and dropped-frame count in diagnostics.

15. Environment variables

Create .env.example:

```
SUPABASE_URL=
SUPABASE_PUBLISHABLE_KEY=
ESP32_IP=
ESP32_PORT=80
CAMERA_INDEX=0
YOLO_MODEL=models/yolo11n.pt
YOLO_CONFIDENCE=0.50
YOLO_HUMAN_HIGH_CONFIDENCE=0.85
YOLO_IMGSZ=640
RUNTIME_MODE=laptop
SENSOR_SIMULATION=true
PRE_EVENT_SECONDS=5
POST_EVENT_SECONDS=8
EVENT_COOLDOWN_SECONDS=10
TEMPORAL_MIN_FRAMES=3
TEMPORAL_WINDOW_SECONDS=1
```

Never commit .env.

16. First demo and acceptance criteria

First demo: start application → camera opens → dashboard appears → operator draws restricted polygon → person enters → YOLO detects → track ID/confidence shown → temporal confirmation → event created → snapshot/video captured → Supabase event/media stored → ESP32 receives alarm → buzzer activates → dashboard shows CRITICAL ALERT.

Also test animal/bird/vehicle, radar-only, ground-only, sensor-fusion simulation, camera blocked, ESP32 offline and Supabase unavailable.

Do not declare DONE merely because source files were generated. Run the application, test it, fix errors, then document what works, what is simulated and what remains future hardware.

17. Required deliverables

Complete runnable source code; ESP32 firmware; laptop application; polygon zone editor; YOLO detection + tracking; temporal validation; sensor fusion; evidence recording; Supabase integration; tests; setup wizard; README; hardware

setup; software setup; ESP32 setup; Supabase setup; testing; troubleshooting; Raspberry Pi migration guide;
PROJECT_STATE.md.